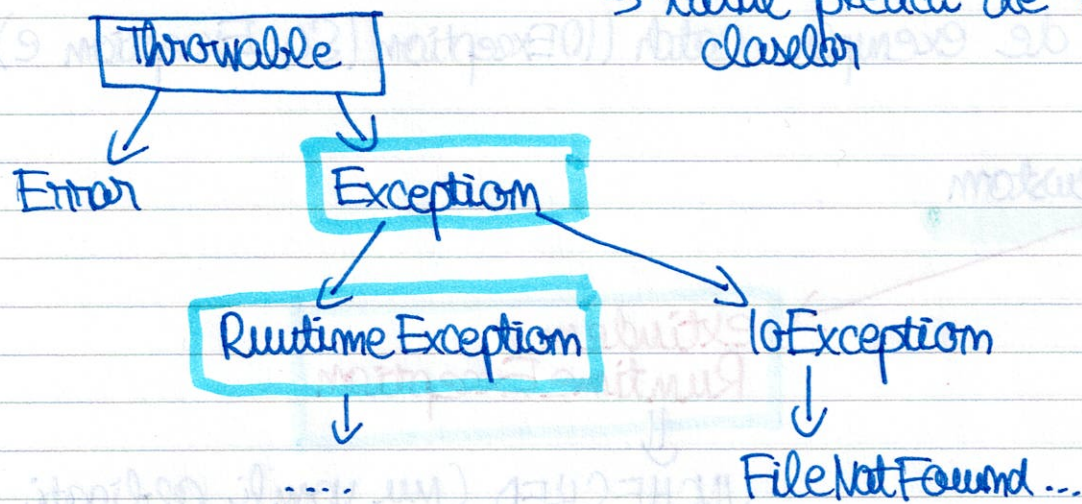


Excepții

① Checked vs. Unchecked Exceptions

→ totul pleacă de la ierarhia claselor



* **Checked Exceptions** - compilatorul ne obligă să le tratăm.

- Sunt situații externe programului nostru pe care le putem anticipa (ex: un fișier nu există, rețeaua a picat)

* **Obligatoriu:** punem codul într-un `try-catch` sau să declarăm metoda cu `throws`.

→ exemple: `IOException`, `FileNotFoundException`...

* **Unchecked Exceptions** - compilatorul nu ne obligă să le tratăm.

- Sunt erori care apar de obicei din cauza unor erori de logică în programare (bug-uri).

* **regulă:** erori care moștenesc `RuntimeException` sau `Error`.

② Try - Catch - Finally

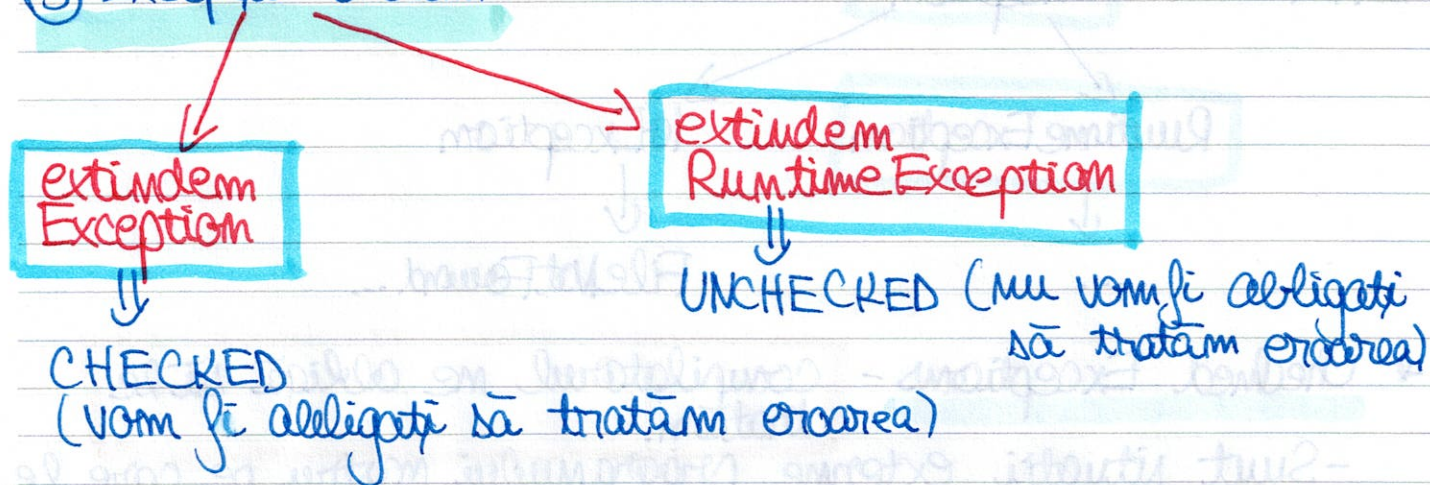
try: codul care poate arunca o eroare

catch: ce se întâmplă când apare excepția

finally: se execută întotdeauna, indiferent dacă a apărut eroare

- Putem avea mai multe locuri catch pt. diferite tipuri de excepții.
- Putem avea mai multe excepții în același catch, folosind de exemplu `catch (IOException | SQLException e)`.

③ Excepții custom



* Best practice: să punem un constructor care primește un mesaj descriptiv și îl trimite la clasa părinte → `super(message)`.

Exemplu: definim o excepție custom CHECKED.

⇒ înseamnă că cine va folosi metoda va fi obligat să trateze eroarea.

```

class LipsaBani extends Exception {
    public LipsaBani(String mesaj) {
        super(mesaj);
    }
}
  
```

→ definiția excepției


```
public class Card {
```

```
    private int sold = 50;
```

la def. metodei se pune
throws....

```
    void plata (int suma) throws LipsaBani {
```

```
        if (suma > sold) {
```

```
            throw new LipsaBani("fonduri insuf.");
```

```
        }
```

```
        if (suma < 0) {
```

```
            throw new RuntimeException("suma gresita");
```

```
        }
```

```
    }
```

```
    sold = sold - suma;
```

```
    System.out.println("Plata ok");
```

```
}
```

```
}
```

```
public class Main {
```

```
    public static void main (String[] args) {
```

```
        Card card = new Card();
```

```
        try {
```

```
            card.plata(100);
```

```
        } catch (LipsaBani e) {
```

```
            System.out.println(e.getMessage());
```

```
        }
```

```
    }
```

dacă am avea mai multe
exceptii personalizate
⇒ le putem da catch
la toate cu
catch(Exception e)

* Dacă avem mai multe exceptii personalizate ⇒ le putem
da catch E

EXEMPLU - exceptii personalizate care mostenesc Exception

LipsaBani.java:

```
package org.example;

public class LipsaBani extends Exception {

    public LipsaBani() {

        super("Aveti prea putini bani, nu se poate face extragerea.");

    }

}
```

SumaPreaMare.java:

```
package org.example;

public class SumaPreaMare extends Exception{

    public SumaPreaMare(){

        super("Suma pe care o adaugati este prea mare!");

    }

}
```

Bancomat.java:

```
package org.example;

public class Bancomat {

    private int sold = 100;

    public void adauga_bani(int suma) throws SumaPreaMare{

        int suma_noua = suma + this.sold;

        if(suma_noua > 1000000){

            throw new SumaPreaMare();

        }

        else{

            this.sold = suma_noua;

            System.out.println("A fost adaugata suma de " + suma + " si acum aveti " + suma_noua);

        }

    }

}
```

```

public void extrage_suma(int suma) throws LipsaBani{

    int suma_noua = this.sold - suma;

    if(suma_noua < 0){

        throw new LipsaBani();

    }

    else{

        this.sold = suma_noua;

        System.out.println("S-a scos suma de " + suma + " si acum aveti " +
suma_noua);

    }

}

}

```

Main.java:

```

package org.example;

public class Main {

    public static void main(String[] args) {

        Bancomat b = new Bancomat();

        // incercam sa adaugam o suma mult prea mare
        try {

            b.adauga_bani(6000000000);

        } catch(Exception e) {

            System.out.println("Exceptie: " + e.getMessage() );

        }

        finally {

            System.out.println("OPERATIA S-A TERMINAT \n \n");

        }

    }

}

```

```
// adaugam o suma care sa nu dea eroare
```

```
try{
    b.adauga_bani(200);
}
catch(Exception e){
    System.out.println("Exceptie: " + e.getMessage() );
}
finally{
    System.out.println("OPERATIA S-A TERMINAT \n \n");
}
```

```
// scoatem o suma prea mare
```

```
try{
    b.extrage_suma(1000);
}
catch(Exception e){
    System.out.println("Exceptie: " + e.getMessage() );
}
finally{
    System.out.println("OPERATIA S-A TERMINAT \n \n");
}
```

```
}
}
```

ce se afiseaza:

Exceptie: Suma pe care o adaugati este prea mare!

OPERATIA S-A TERMINAT

A fost adaugata suma de 200 si acum aveti 300

OPERATIA S-A TERMINAT

Exceptie: Aveti prea putini bani, nu se poate face extragerea.

OPERATIA S-A TERMINAT